

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP

Project: "Review sense ---- Extracting feedback from customer reviews"

Milestone 1: User authentication and Profile management

SUBMITTED BY:

Name: **RITAM DAS**

Domain: **ARTIFICIAL INTELLIGENCE**(Batch-4)

Date: 01.11.25

1. INTRODUCTION

The project “Review Sense – Extracting Insights from Customer Feedback” aims to develop an AI-based system capable of understanding and analyzing customer opinions from online reviews.

The main purpose of the model is to identify the sentiment behind customer feedback — whether it is positive, negative, or neutral — and generate useful insights that can help businesses improve their services and customer experience. The system functions by processing textual data, cleaning it, and applying natural language techniques to classify sentiments accurately.

The goal of Milestone 1 was to build a secure and user-friendly foundation for the application by implementing User Authentication and Profile Management using FastAPI, Streamlit, and SQLite, ensuring that users can safely register, log in, and manage their personal information within the platform.

2. OBJECTIVE

User Registration & Login – Develop a secure system where users can register and log in using unique credentials, ensuring authorized access to the application.

Profile Management – Allow users to create and update personal details within their profile.

Forgot Password Functionality – Implement an option for users to reset their password securely if forgotten.

Database Integration – Use **SQLite** to store user data and profile information efficiently and reliably.

Frontend–Backend Connection – Integrate **Streamlit** (frontend) with **FastAPI** (backend) for smooth data exchange and user interaction.

3. TECH STACK

The development of Milestone 1 utilized the following technologies and frameworks:

- Frontend: Streamlit – for building the interactive user interface for registration, login, and profile management.
- Backend: FastAPI – for creating secure and efficient API endpoints and managing authentication logic.
- Database: SQLite 3 – lightweight relational database to store user details and profile data.
- Programming Language: Python 3.11 – used for both backend and frontend scripting.
- Libraries & Tools:
 - bcrypt – for password encryption.
 - jwt – for session authentication tokens.
 - requests – for API communication between FastAPI and Streamlit.
 - os and sqlite3 – for database connectivity and file handling.
- Version Control: Git & GitHub – for source-code management and collaboration.
- IDE: Visual Studio Code – for coding, testing, and debugging.

4. CODE IMPLEMENTATION

The Milestone 1 implementation was carried out using FastAPI as the backend framework and Streamlit as the frontend. The following components were implemented:

DATABASE SETUP : The database is implemented using SQLite, and ORM (Object Relational Mapping) is handled by SQLAlchemy. The code below initializes the database engine, creates the session, and defines the schema for the User and PasswordReset tables.

```
from sqlalchemy import create_engine, Column, Integer, String, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from datetime import datetime
```

```
DATABASE_URL = "sqlite:///./users.db"
engine = create_engine(DATABASE_URL,
connect_args={"check_same_thread": False})
Base = declarative_base()
SessionLocal = sessionmaker(bind=engine)
db = SessionLocal()
```

```
# User model
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True)
    password = Column(String)
    full_name = Column(String, default="")
    email = Column(String, default="")
    security_question = Column(String, default="")
    security_answer_hash = Column(String, default="")
    age_group = Column(String, default="")
    language_preference = Column(String, default="")
    wellness_goals = Column(String, default="")
```

```
# Password Reset model
class PasswordReset(Base):
    __tablename__ = "password_resets"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, index=True)
    token = Column(String, unique=True, index=True)
    expires_at = Column(DateTime)
```

```
Base.metadata.create_all(bind=engine)
```

FastAPI Setup and Request Models : The FastAPI framework is used to create RESTful APIs for handling user actions such as registration, login, and password change.

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
# Request Models
```

```
class RegisterRequest(BaseModel):
    username: str
    password: str
    full_name: str | None = None
    email: str | None = None
    security_question: str | None = None
    security_answer: str | None = None
```

```
class LoginRequest(BaseModel):
    username: str
    password: str
```

```
class ChangePasswordRequest(BaseModel):
    username: str
    old_password: str
    new_password: str
```

```
class ProfileUpdateRequest(BaseModel):
    full_name: str | None = None
    email: str | None = None
    age_group: str | None = None
```

Explanation:

- Each request model defines the structure of incoming API data.
- FastAPI automatically validates these fields before processing requests.

User Registration and Login :

This section allows users to register and log into the system.

```
from fastapi import HTTPException
import hashlib
```

```
def hash_text(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

```
@app.post("/register")
def register_user(request: RegisterRequest):
    user_exists = db.query(User).filter(User.username ==
request.username).first()
    if user_exists:
        raise HTTPException(status_code=400, detail="Username already
exists")
```

```
    security_answer_hash = hash_text(request.security_answer or "")
    new_user = User(
        username=request.username,
```

```

        password=request.password,
        email=request.email or "",
        full_name=request.full_name or "",
        security_question=request.security_question or "",
        security_answer_hash=security_answer_hash
    )
    db.add(new_user)
    db.commit()
    return {"message": "Registration successful"}

@app.post("/login")
def login_user(request: LoginRequest):
    user = db.query(User).filter(
        User.username == request.username,
        User.password == request.password
    ).first()
    if not user:
        raise HTTPException(status_code=400, detail="Invalid credentials")
    return {"message": f"Welcome {request.username}"}

```

Explanation:

- Passwords and security answers are hashed using SHA-256.
- Registration checks for duplicate usernames.
- Invalid credentials trigger a clear error message

Profile Management :

Users can view or update their profile data.

```

@app.get("/profile/{username}")
def get_profile(username: str):
    user = db.query(User).filter(User.username == username).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return {
        "username": user.username,
        "email": user.email,
        "full_name": user.full_name,
        "language_preference": user.language_preference
    }

```

```

@app.put("/profile/{username}")
def update_profile(username: str, request: ProfileUpdateRequest):
    user = db.query(User).filter(User.username == username).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

```

```

if request.email:
    user.email = request.email
if request.full_name:
    user.full_name = request.full_name
if request.wellness_goals:
    user.wellness_goals = request.wellness_goals

db.commit()
return {"message": "Profile updated successfully"}

```

Password Reset Functionality :

Implements multi-step password recovery using security questions and tokens.

```

from datetime import datetime, timedelta
import uuid

@app.post("/forgot_password/start")
def forgot_password_start(username: str):
    user = db.query(User).filter(User.username == username).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return {"security_question": user.security_question}

@app.post("/forgot_password/verify")
def forgot_password_verify(username: str, answer: str):
    user = db.query(User).filter(User.username == username).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    if user.security_answer_hash != hash_text(answer):
        raise HTTPException(status_code=400, detail="Incorrect security answer")

    token = str(uuid.uuid4())
    expires_at = datetime.utcnow() + timedelta(minutes=10)
    db.add>PasswordReset(username=username, token=token,
expires_at=expires_at))
    db.commit()
    return {"token": token, "expires_at": expires_at}

```

Data Export Feature: The backend allows exporting user data to different formats for analysis.

```

import pandas as pd
from fastapi.responses import FileResponse, JSONResponse

```

```

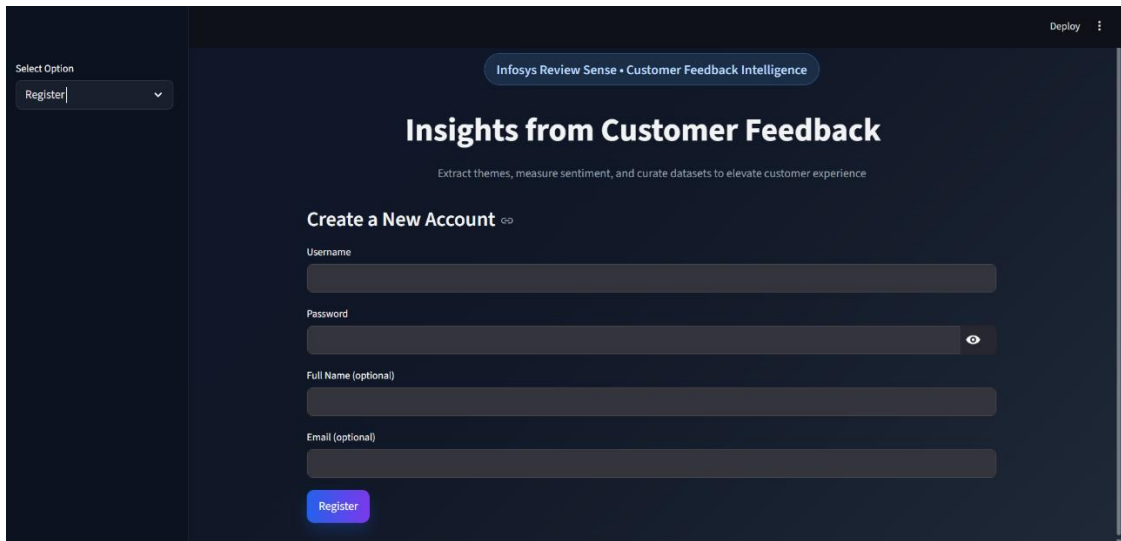
@app.get("/export/users/csv")
def export_users_csv():
    df = pd.read_sql_query("SELECT * FROM users", db.bind.connect())
    csv_filename = "users_export.csv"
    df.to_csv(csv_filename, index=False)
    return FileResponse(csv_filename, media_type="text/csv")

@app.get("/export/users/json")
def export_users_json():
    users = db.query(User).all()
    data = [{"id": u.id, "username": u.username, "email": u.email} for u in users]
    return JSONResponse(content=data)

```

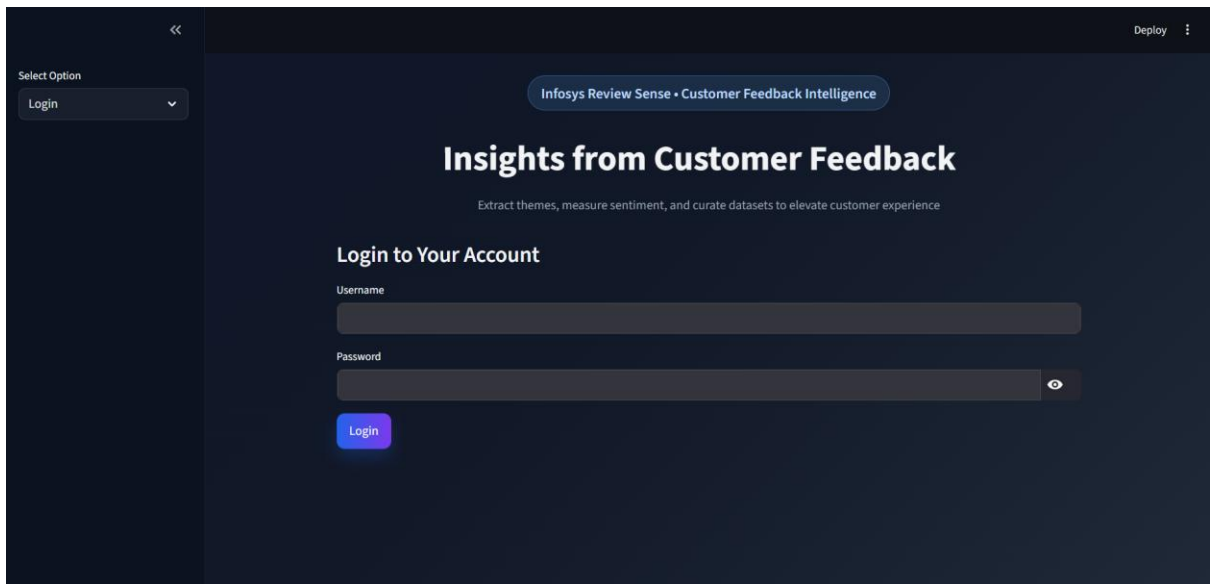
5. Screenshots and Output

Below are the screenshots demonstrating the functionality of each component of Milestone 1.



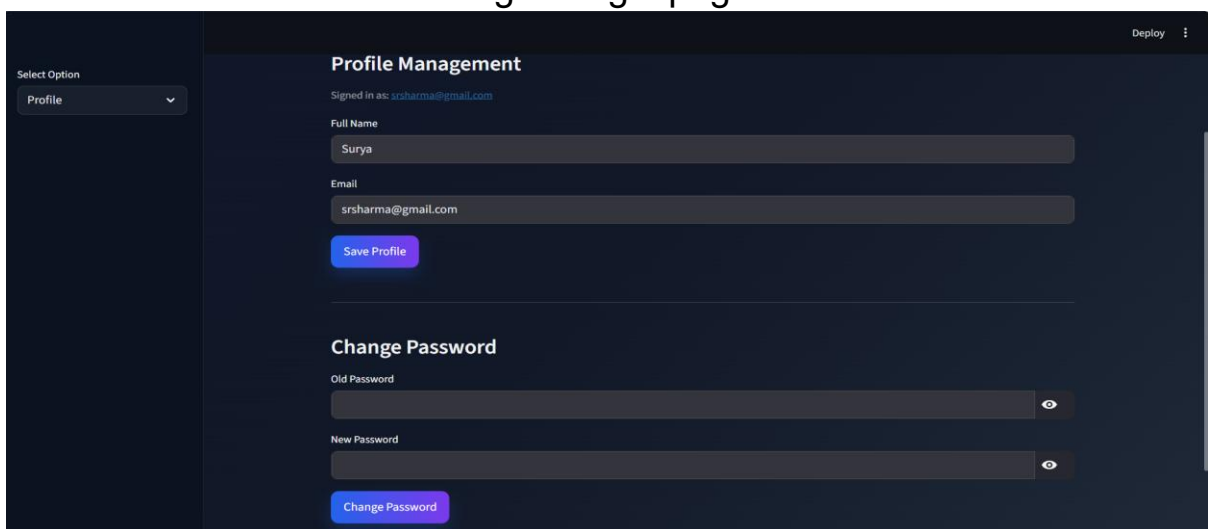
The screenshot shows a web application interface for 'Infosys Review Sense - Customer Feedback Intelligence'. The main heading is 'Insights from Customer Feedback' with a subtitle 'Extract themes, measure sentiment, and curate datasets to elevate customer experience'. On the left, there is a sidebar with a 'Select Option' dropdown menu currently set to 'Register'. The main content area is titled 'Create a New Account' and contains a registration form with the following fields: 'Username', 'Password' (with a toggle icon), 'Full Name (optional)', and 'Email (optional)'. A blue 'Register' button is located at the bottom of the form. In the top right corner, there is a 'Deploy' button with a three-dot menu icon.

Fig1: Registration Page



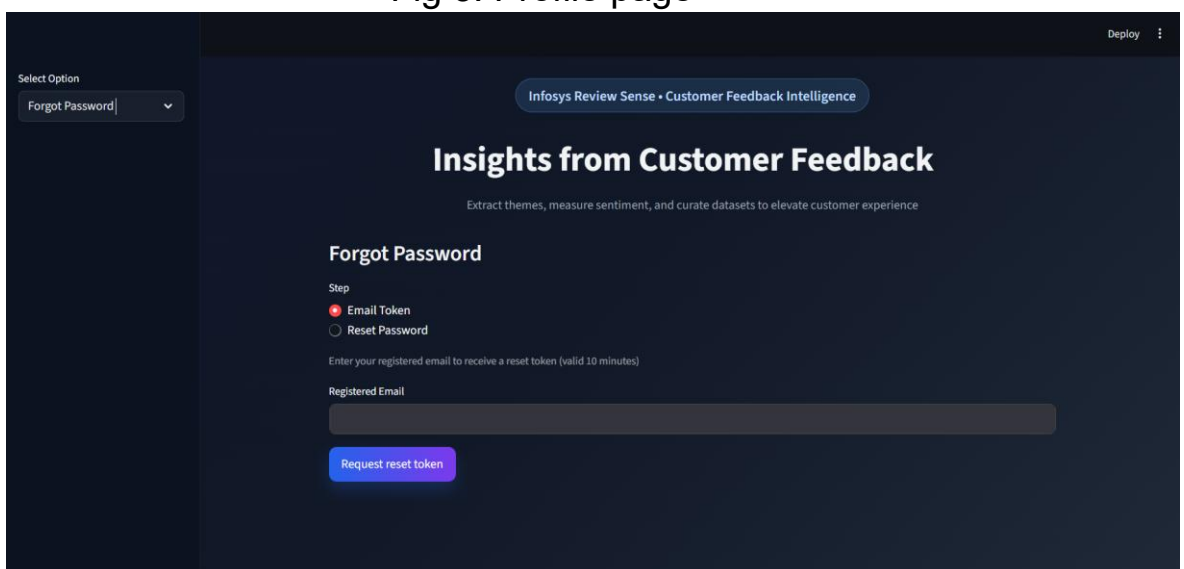
The login page features a dark blue background. On the left, a sidebar contains a 'Select Option' dropdown menu with 'Login' selected. The main content area has a header with 'Infosys Review Sense • Customer Feedback Intelligence' and a title 'Insights from Customer Feedback' with a subtitle 'Extract themes, measure sentiment, and curate datasets to elevate customer experience'. Below this is a 'Login to Your Account' section with input fields for 'Username' and 'Password' (with a toggle icon), and a blue 'Login' button.

Fig 2: Login page



The profile management page has a dark blue background. The sidebar shows 'Select Option' with 'Profile' selected. The main content area has a title 'Profile Management' and a subtitle 'Signed in as: srsharma@gmail.com'. It contains two sections: 'Full Name' with a text input field showing 'Surya' and a blue 'Save Profile' button, and 'Change Password' with input fields for 'Old Password' and 'New Password' (each with a toggle icon) and a blue 'Change Password' button.

Fig 3: Profile page



The forgot password page has a dark blue background. The sidebar shows 'Select Option' with 'Forgot Password' selected. The main content area has a header with 'Infosys Review Sense • Customer Feedback Intelligence' and a title 'Insights from Customer Feedback' with a subtitle 'Extract themes, measure sentiment, and curate datasets to elevate customer experience'. Below this is a 'Forgot Password' section with a 'Step' indicator showing 'Email Token' selected (with a red dot) and 'Reset Password' (with a grey dot). A subtitle reads 'Enter your registered email to receive a reset token (valid 10 minutes)'. There is an input field for 'Registered Email' and a blue 'Request reset token' button.

Fig 4: Forgot password page

id	username	password	full_name	email	security_question	security_answer_hash	age_group	language_preference	wellness_goals
1	Ritam123	rit5532	NONE	NONE	NONE	NONE	NONE	NONE	NONE
2	Ritam743	rit3321	NONE	NONE	NONE	NONE	NONE	NONE	NONE
3	Ritam7434	rit3321	NONE	NONE	NONE	NONE	NONE	NONE	NONE
4	Ritam855	rit874395	NONE	NONE	NONE	NONE	NONE	NONE	NONE
5	Ritam885	Ritam438	NONE	NONE	NONE	NONE	NONE	NONE	NONE
6	Ritam998	Rit5489	NONE	NONE	NONE	NONE	NONE	NONE	NONE
7	Ritam765	Rit34889	NONE	NONE	NONE	NONE	NONE	NONE	NONE
8	Ritam6654	Rit88765	NONE	NONE	NONE	NONE	NONE	NONE	NONE
9	Ritam98	d028ritam7	Ritam	ritamd289@gmail.com					
10	Ritam7665	Rit45889	Ritam Das	rdas8899@gmail.com					
11	Ritam5643	Rit86790	Rohit	rdhaase@gmail.com					
12	Rit6779	Gd89908	Surya	sreharma@gmail.com					

Fig 5: Database page

6. Conclusion

Milestone 1 laid the foundation for the complete Review Sense application. Through this phase, the backend and frontend were successfully connected, user data was securely stored, and a strong authentication framework was established. This sets the stage for upcoming AI functionalities, where the system will analyze customer reviews to generate meaningful insights using natural language processing techniques.

7. Acknowledgment

I would like to express my gratitude to the Infosys Springboard team and my mentor for their continuous guidance and support during the development of this milestone. This internship has been a valuable experience in learning full-stack development, authentication systems, and the integration of AI in real-world applications